

TutorMatch AI

ระบบจับคู่ตัวเตอร้อัจฉริยะด้วยปัญญาประดิษฐ์ ผ่าน LINE Official Account

ประเภทระบบ: Web Application (LINE OA & LIFF Integration)

เทคโนโลยีหลัก: Python, FastAPI, DeepFace, Sentence-Transformers, PostgreSQL

ผู้พัฒนา: Antigravity AI Assistant

เอกสาร: แนวคิดระบบ สถาปัตยกรรม และขั้นตอนการดำเนินงาน (Concept & Architecture Design)

บทสรุปผู้บริหาร (Executive Summary):

TutorMatch AI คือระบบจับคู่ตัวเตอร้อัจฉริยะที่อำนวยความสะดวกให้ผู้เรียนสามารถค้นหาตัวเตอรที่เหมาะสมที่สุดได้ผ่าน LINE โดยมีจุดเด่นในเรื่องความปลอดภัยจากการยืนยันตัวตนด้วยใบหน้า (Face Verification) ผ่าน LINE LIFF และกลไกการจับคู่ด้วย AI ประมวลผลภาษาธรรมชาติเชิงความหมาย (Semantic Matching) ช่วยให้จับคู่ตัวเตอรได้ตรงตามทักษะ ความรู้ ความถนัด และรูปแบบการเรียนรู้ที่นักเรียนต้องการอย่างแท้จริง

1. แนวคิดและที่มาของโครงการ (System Concept)

ในระบบการเรียนการสอนพิเศษในปัจจุบัน ปัญหาที่มักพบเจอคือความไม่เหมาะสมกันระหว่างสไตล์การสอนของติวเตอร์และพฤติกรรมการเรียนรู้ของนักเรียน รวมถึงความปลอดภัยในการดำเนินกิจกรรม ซึ่งระบบ TutorMatch AI ถูกสร้างขึ้นเพื่อแก้ไขปัญหาด้วยองค์ประกอบ 2 ด้านหลัก:

• ระบบการยืนยันตัวตนด้วยใบหน้า (Face Verification)

ก่อนการใช้งานหรือลงทะเบียนเพื่อความปลอดภัย ทั้งติวเตอร์และนักเรียนจะต้องทำการลงทะเบียนใบหน้าผ่านกล้องหน้าของ LINE LIFF จากนั้นทุกครั้งที่มีการเข้าใช้งานฟังก์ชันสำคัญ ระบบจะทำการสแกนใบหน้าและแปลงค่าเป็น Face Embedding เพื่อนำมาเปรียบเทียบกับฐานข้อมูล เป็นการยืนยันความปลอดภัยและตัวตนจริงของผู้ใช้งานในระบบ

• ระบบการจับคู่เชิงความหมาย (Semantic Matching Engine)

แตกต่างจากระบบแบบเก่าที่ค้นหาจากคำสำคัญ (Keywords) เท่านั้น เนื่องจากผู้เรียนบางคนอาจเขียนรายละเอียดเป็นภาษาธรรมชาติที่เป็นความต้องการจริงๆ เช่น 'หาพี่สอนเลขใจดี เน้นปูพื้นฐานตั้งแต่ต้น ไม่ดูเป็นกันเอง' ระบบจะใช้ AI ในการถอดความหมายของความต้องการดังกล่าว แล้วจับคู่กับประวัติ ทักษะ ความถนัด และจุดเด่นของติวเตอร์ที่บันทึกไว้ในระบบ เพื่อคำนวณเป็น Matching Score ที่ดีที่สุด

2. สถาปัตยกรรมและการไหลของข้อมูล (System Architecture)

สถาปัตยกรรมระบบถูกออกแบบเป็น Micro-component ที่เชื่อมโยงเข้ากับระบบนิเวศของ LINE ดังนี้:

ส่วนประกอบ (Component)	หน้าที่การทำงาน (Responsibility)
LINE OA & Webhook	เป็นช่องทางรับส่งข้อความหลัก ส่งข้อมูลกิจกรรมจากห้องแชท LINE ไปยังเซิร์ฟเวอร์หลังบ้าน
LINE LIFF (Frontend)	หน้าจอเว็บฝังใน LINE สำหรับหน้ากรอกฟอร์ม สมัครสมาชิก และเรียกใช้กล้องหน้าสำหรับสแกนใบ
FastAPI Backend	ระบบประมวลผลหลัก (API Gateway) จัดการสิทธิ์การใช้งาน บันทึกข้อมูล และประสานงานกับโมดูล
AI Engines	1) Face Verification (ประมวลรูปภาพด้วย DeepFace) 2) Matching Engine (จับคู่ข้อความ
Database (PostgreSQL)	จัดเก็บข้อมูลติวเตอร์ ข้อมูลนักเรียน รวมถึงเวกเตอร์ของใบหน้าและประวัติติวเตอร์ (pgvector)

3. เทคโนโลยีที่เลือกใช้ (Technology Stack)

- Backend Web Framework: Python 3.10+, FastAPI, Uvicorn, Requests (ยืดหยุ่น รวดเร็ว รองรับการงาน Async)
- LINE Integration: LINE Messaging API SDK, LINE LIFF (HTML5/CSS/JavaScript สำหรับหน้าจอเว็บย่อย)
- Face Recognition (AI): DeepFace, OpenCV, MediaPipe (สกัดลักษณะใบหน้าและวัดระยะห่างความเหมือน)
- Natural Language Processing: Sentence-Transformers, PyThaiNLP, Gemini API (สำหรับประมวลภาษาไทยและทำ Semantic Matching)
- Database System: PostgreSQL พร้อม Extension pgvector หรือ SQLite (สำหรับช่วงเริ่มต้นพัฒนา)

4. กลไก AI และอัลกอริทึม (AI Engine Logic)

• ขั้นตอนการยืนยันตัวตนด้วยใบหน้า:

- ผู้ถ่ายภาพใบหน้าสดจากกล้องบน LINE LIFF ส่งไปยัง FastAPI Backend
- API ส่งภาพให้โมเดล DeepFace สกัดเวกเตอร์ลักษณะใบหน้าออกมา (128 หรือ 512 มิติ)
- เปรียบเทียบเวกเตอร์ใหม่กับเวกเตอร์เดิมในฐานข้อมูลด้วยระยะห่าง Cosine Distance
- หากระยะห่างน้อยกว่าเกณฑ์ที่กำหนด (เช่น < 0.40) ถือว่าผ่านการตรวจสอบยืนยันตัวตน

```
# code snippet: การเปรียบเทียบใบหน้าในหลังบ้าน
from deepface import DeepFace

def verify_user_face(registered_img, current_img):
    # เรียกใช้ DeepFace เปรียบเทียบเวกเตอร์สองภาพ
    result = DeepFace.verify(
        img1_path=registered_img,
        img2_path=current_img,
        model_name='ArcFace'
    )
    return result['verified'], result['distance']
```

• ขั้นตอนการจับคู่ตัวเตอร์เชิงความหมาย:

- สร้างเวกเตอร์โปรไฟล์: ระบบแปลงข้อมูลของตัวเตอร์เป็น Vector Embedding เก็บใน PostgreSQL (pgvector)
- ประมวลเวกเตอร์ความต้องการ: นักเรียนกรอกข้อมูลความต้องการและถูกแปลงเป็นเวกเตอร์ด้วย Sentence-Transformer ตัวเดียวกัน
- เปรียบเทียบคะแนนความเหมือน: ค้นหาและดึงประวัติตัวเตอร์ที่มีค่าความใกล้เคียงสูงสุด 3 อันดับแรก ด้วยคำสั่ง Cosine Similarity
- การคัดกรองเบื้องต้น (Hard Filtering): กรองเฉพาะตัวเตอร์ที่วิชาสอนตรงและมีเวลาว่างตรงกับงบประมาณของนักเรียนก่อนคำนวณ

5. ขั้นตอนการพัฒนาาระบบทีละขั้น (Implementation Plan)

ขั้นตอนที่ 1: การตั้งค่าโครงการและการจัดเตรียมสิ่งแวดล้อม

ติดตั้งโปรแกรมและไลบรารีที่จำเป็น (FastAPI, PyThaiNLP, DeepFace)
พร้อมเตรียมบัญชีผู้ใช้ LINE Developers เพื่อรับ Token และเตรียมสร้าง Channel Webhook ในส่วนควบคุม

ขั้นตอนที่ 2: พัฒนาระบบยืนยันตัวตนด้วยใบหน้า

เขียนกลองหน้าบน LINE LIFF โดยให้เว็บของ LIFF สามารถจับภาพใบหน้าส่งเป็น Base64 หรืออัปโหลดไฟล์ และเขียน API ฟังก์ชัน Python เพื่อทำการเปรียบเทียบรูปภาพกับฐานข้อมูลในลักษณะของ Metric Learning

ขั้นตอนที่ 3:

พัฒนาระบบจัดการประวัติตัวเตอร์และเก็บข้อมูลความเชี่ยวชาญ

สร้างหน้าเว็บฟอร์มให้ตัวเตอร์กรอกข้อมูลความถนัด และประวัติต่างๆ
จากนั้นฝั่งหลังบ้านจะดึงข้อมูลนี้มาผ่านกระบวนการ Embeddings
เพื่อเปลี่ยนเป็นข้อมูลเชิงตัวเลขแล้วบันทึกลงสู่ฐานข้อมูล

ขั้นตอนที่ 4: สร้างหน้าจอสำหรับนักเรียน และเขียน AI Matching Engine

พัฒนาหน้าจอการบ่อนความต้องการของนักเรียน และโมดูล AI
ที่ทำการค้นหาเวกเตอร์ที่ใกล้เคียงที่สุด จากนั้นระบบจะแสดงผลตัวเตอร์ที่แนะนำเป็น
Flex Message ผ่านทางแชท LINE OA เพื่อให้ติดต่อพูดคุยตกลงเรียนกันต่อไป